

Phân tích và khai phá dữ liệu văn bản

Chương 2: Xử lý dữ liệu văn bản

GV: TS. Nguyễn Thị Mai Trang

Email: trangntm@ptit.edu.vn

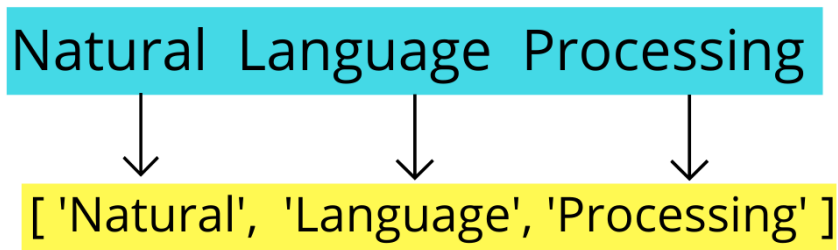
Nội dung

- Tách từ (Tokenization)
- Chuẩn hóa văn bản (Text Normalization)
- Hiểu cấu trúc ngữ pháp (Syntactic Analysis)

1. Tách từ (Tokenization)

- **Khái niệm:** Tách từ là quá trình chia một chuỗi văn bản thành các đơn vị nhỏ hơn có ý nghĩa, gọi là "token".
 - Token là gì? Token thường là từ, nhưng cũng có thể là số, dấu câu, hoặc các đơn vị có ý nghĩa khác tùy thuộc vào mục đích.
- **Tầm quan trọng:** Đây là bước đầu tiên và cơ bản nhất trong hầu hết các tác vụ xử lý ngôn ngữ tự nhiên (NLP). Nếu tách từ không chính xác, các bước xử lý sau sẽ bị ảnh hưởng.

Tokenization



1.1 Các phương pháp tách từ

- **Tách theo khoảng trắng:** Đơn giản nhất, chia chuỗi theo dấu cách.
 - Ưu điểm: Dễ thực hiện.
 - Nhược điểm: Không xử lý được dấu câu dính liền từ, từ ghép, hoặc các ngôn ngữ không có khoảng trắng rõ ràng (ví dụ: tiếng Trung, tiếng Nhật).
- **Tách theo dấu câu:** Chia chuỗi theo các dấu câu (dấu phẩy, chấm, hỏi, than...).
- **Tách dựa trên từ điển:** Sử dụng một từ điển các từ đã biết để nhận diện và tách các từ.
- **Tách dựa trên quy tắc:** Áp dụng các quy tắc ngữ pháp hoặc hình thái học để tách từ (ví dụ: tách tiền tố, hậu tố).
- **Tách dựa trên học máy:** Sử dụng các mô hình học máy để dự đoán ranh giới từ.

1.2 Ví dụ

- "Hôm nay, thời tiết thật đẹp! Tôi rất vui."
- Tách theo khoảng trắng đơn giản:
 - ["Hôm", "nay,", "thời", "tiết", "thật", "đẹp!", "Tôi", "rất", "vui."]
 - Nhận xét: Dấu phẩy và dấu chấm than vẫn dính vào từ.
- Tách nâng cao (có xử lý dấu câu):
 - ["Hôm", "nay", ",", "thời", "tiết", "thật", "đẹp", "!", "Tôi", "rất", "vui", "."]
 - Nhận xét: Các dấu câu đã được tách riêng thành các token.
- Ví dụ với từ ghép (tiếng Việt):
 - "Chính phủ Việt Nam đã ban hành chính sách mới."
 - Tách từ mong muốn: ["Chính phủ", "Việt Nam", "đã", "ban hành", "chính sách", "mới", "."]
 - Thách thức: "Chính phủ", "Việt Nam", "ban hành", "chính sách" là các từ ghép có ý nghĩa riêng, không nên tách rời. Vậy tách như thế nào?

1.3 Cách tiếp cận dựa trên từ điển

► Phương pháp Longest matching

- Duyệt câu đầu vào tuần tự và chọn từ **dài nhất** nếu từ đó **có trong từ điển**
- Đây là một kỹ thuật phổ biến trong xử lý chuỗi, đặc biệt trong bài toán **tách từ** trong các ngôn ngữ không có dấu cách giữa các từ (như tiếng Trung, tiếng Nhật, tiếng Việt).
- Ví dụ
 - Input: Học sinh học sinh học
 - *Duyệt tuần tự từ trái qua phải*
 - Output: Học_sinh học_sinh học

1.3 Cách tiếp cận dựa trên từ điển

► Phương pháp Maximum matching

- Sử dụng từ điển, **tạo ra tất cả các cách tách từ** có thể cho một câu bất kỳ, sau đó câu được tách từ đúng được chọn là câu chứa **ít từ nhất**
- Ví dụ
 - Input: Đó là cách để tuyên truyền thông tin
 - Output: Đó là cách để tuyên_truyền thông_tin

1.3 Cách tiếp cận dựa trên từ điển

► Phương pháp Maximum matching

- Sử dụng từ điển, **tạo ra tất cả các cách tách từ** có thể cho một câu bất kỳ, sau đó câu được tách từ đúng được chọn là câu chứa **ít từ nhất**

Ví dụ: "học bổng đại học"

Từ điển: {"học", "bổng", "học bổng", "đại học", "học đại học"}.

Các cách tách có thể có:

"học bổng | đại học"

"học | bổng | đại học"

"học bổng | đại học"  (Chỉ gồm 2 từ, tối ưu nhất)

"học | bổng | đại học"  (Có 3 từ, không tối ưu vì số lượng từ nhiều hơn)

1.3 Cách tiếp cận dựa trên từ điển

► Ưu điểm

- Đơn giản, dễ cài đặt

► Nhược điểm

- Phụ thuộc vào từ điển
- Độ chính xác không cao
- Không giải quyết được trường hợp nhập nhằng

2. Chuẩn hóa văn bản

- **Khái niệm:** Chuẩn hóa văn bản là tập hợp các kỹ thuật nhằm đưa văn bản về một dạng thống nhất, dễ dàng xử lý hơn cho các tác vụ NLP tiếp theo.
- **Mục tiêu:**
 - Giảm thiểu sự đa dạng của dữ liệu (ví dụ: "Apple" và "apple" thành "apple").
 - Loại bỏ nhiễu và thông tin không cần thiết.
 - Cải thiện hiệu suất của các mô hình học máy.
- **Các bước chính:** Làm sạch, tách từ, loại bỏ từ dừng, chỉnh sửa lỗi chính tả, lấy từ gốc.

2.1 Làm sạch văn bản (Text Cleaning)

- **Mục tiêu:** Loại bỏ các yếu tố không mong muốn hoặc không mang ý nghĩa ngữ nghĩa từ văn bản.
- **Các loại nhiễu thường gặp:**
 - HTML tags, XML tags
 - URLs (địa chỉ web)
 - Ký tự đặc biệt, biểu tượng cảm xúc (emojis)
 - Số (tùy ngữ cảnh)
 - Khoảng trắng thừa (multiple spaces, leading/trailing spaces)
 - Ký tự không phải ASCII (nếu cần)

Ví dụ

- Văn bản gốc: "Sản phẩm này
 thật tuyệt vời! Mua ngay tại đây. Giá chỉ 99\$! 😊"

- Các bước làm sạch:

1. Loại bỏ HTML tags: "Sản phẩm này thật tuyệt vời! Mua ngay tại 'http://example.com' đây. Giá chỉ 99\$! 😊"
2. Loại bỏ URLs: "Sản phẩm này thật tuyệt vời! Mua ngay tại đây. Giá chỉ 99\$! 😊" (Nếu URL là văn bản, cần regex để phát hiện)
3. Loại bỏ ký tự đặc biệt/emojis: "Sản phẩm này thật tuyệt vời Mua ngay tại đây Giá chỉ 99"
4. Loại bỏ số (tùy chọn): "Sản phẩm này thật tuyệt vời Mua ngay tại đây Giá chỉ"
5. Chuyển về chữ thường: "sản phẩm này thật tuyệt vời mua ngay tại đây giá chỉ"
6. Xóa khoảng trắng thừa: "sản phẩm này thật tuyệt vời mua ngay tại đây giá chỉ"

Văn bản sau làm sạch (tùy thuộc vào quy tắc): "sản phẩm này thật tuyệt vời mua ngay tại đây giá chỉ"

2.2 Tách từ tiếng Việt

- Là quá trình chia văn bản thành các đơn vị nhỏ hơn (token) (*nhắc lại*)
- **Tầm quan trọng trong chuẩn hóa:** Sau khi làm sạch, việc tách từ chính xác là cực kỳ quan trọng để các bước tiếp theo (loại bỏ từ dừng, lấy từ gốc) hoạt động hiệu quả.
- Thách thức với tiếng Việt:
- Từ ghép: "học sinh", "máy tính xách tay".
- Từ viết tắt: "TP.HCM", "CNTT".
- Dấu câu: Dính liền với từ ("điểm.", "vui!").
- Số và đơn vị: "10kg", "50km/h".

2.2 Tách từ tiếng Việt

```
import re

def tokenize_with_punct(text):
    # Regex: tách dấu câu riêng biệt
    # (?=\W) : trước ký tự không phải chữ
    # (?<=\W): sau ký tự không phải chữ
    tokens = re.findall(r"\w+|[\^\w\s]", text, re.UNICODE)
    return tokens

# Ví dụ
s = "Hôm nay trời đẹp quá!"
print(tokenize_with_punct(s))
# 🖱 ['Hôm', 'nay', 'trời', 'đẹp', 'quá', '!']

s2 = "Tôi đạt 9 điểm."
print(tokenize_with_punct(s2))
# 🖱 ['Tôi', 'đạt', '9', 'điểm', '.']
```

Ví dụ với tiếng Việt

- "TP.HCM là thành phố lớn nhất Việt Nam, với hơn 9 triệu dân (số liệu 2023)."
- Tách từ thủ công/mong muốn:
- ["TP.HCM", "là", "thành phố", "lớn nhất", "Việt Nam", ",", "với", "hơn", "9", "triệu", "dân", "(", "số liệu", "2023", ")", "."]
- Công cụ hỗ trợ (ví dụ: Underthesea trong Python)

2.3 Loại bỏ ký tự đặc biệt và từ dừng

- **Từ dừng (stop words)** là các từ rất phổ biến trong ngôn ngữ nhưng thường không mang nhiều ý nghĩa phân biệt hoặc ngữ nghĩa quan trọng cho các tác vụ NLP
(ví dụ: "là", "và", "của", "một", "những").
- **Mục tiêu:**
 - Giảm kích thước dữ liệu.
 - Giảm thời gian xử lý.
 - Cải thiện hiệu suất của mô hình bằng cách loại bỏ nhiễu.
- **Danh sách từ dừng:** Thường có sẵn cho từng ngôn ngữ (ví dụ: NLTK cho tiếng Anh, các thư viện tiếng Việt). Có thể tùy chỉnh.

2.3 Loại bỏ ký tự đặc biệt và từ dừng

- Loại bỏ các ký tự không phải chữ cái hoặc số (nếu không cần thiết) để giảm nhiễu.
- Các ký tự thường bị loại bỏ: !, @, #, \$, %, ^, &, *, (,), -, _, +, =, {, }, [,], |, \, ;, :, ', ", <, >, ?, /, .., ,, ~
- **Lưu ý:** Cần cân nhắc ngữ cảnh. Ví dụ: trong phân tích cảm xúc, dấu chấm than ! có thể mang ý nghĩa tích cực mạnh. Trong phân tích mã, \$, _ lại có ý nghĩa.
- Ví dụ:
 - Văn bản gốc: "Sản phẩm này rất tốt!!!"
 - Sau khi loại bỏ: "Sản phẩm này rất tốt"

2.3 Ví dụ

- Danh sách từ dừng tiếng Việt (ví dụ một phần):

"là", "và", "của", "một", "những", "có", "đã", "sẽ", "không", "rất", "này", "kia", "đó",
"thì", "mà", "được", "với", "trong", "trên", "dưới", "về", "cho", "để", "như", "hay",
"vẫn", "cũng", "chỉ", "từng", "mỗi", "mọi", "cả", "nào", "gì", "ai", "đâu", "khi",
"lúc", "vừa", "mới", "còn", "hết", "thêm", "bớt", "ít", "nhiều", "hơn", "kém", "nhất",
"quá", "lắm", "thật", "chắc", "có thể", "không thể", "nên", "phải", "đừng", "chớ",
"hãy", "xin", "mong", "muốn", "cần", "phải", "được", "bị", "làm", "cho", "để", "mà",
"thì", "là", "vậy", "như", "thế", "nào", "đó", "đây", "kia", "ấy", "chúng tôi", "chúng
ta", "tôi", "bạn", "anh", "chị", "em", "ông", "bà", "cô", "chú", "dì", "cháu", "họ",
"người", "ta", "mình", "người ta", "nó", "chúng", "tụi", "họ", "ấy", "đó", "đây",
"kia", "v.v."

- Văn bản gốc: "**Đây là một cuốn sách rất hay và tôi đã đọc nó hai lần.**"
- Sau khi loại bỏ từ dừng: "**cuốn sách hay đọc hai lần**"
- Nhận xét: Văn bản ngắn gọn hơn, giữ lại các từ khóa mang ý nghĩa chính.

Bài tập 1

1) Hãy viết code trên python để tách câu sau thành các token theo 3 cách khác nhau: longest matching, maximum matching và cách hiện đại sử dụng VNCoreNLP

- “Hôm nay là thứ Tư. Chúng tôi có tiết học về phân tích và xử lý dữ liệu văn bản. Tiết học khá thú vị tuy nhiên vẫn còn nhiều thứ chưa hiểu hết.”

2) Loại bỏ từ dừng trong các kết quả đã thu được từ 3 cách trên.

2.4. Chỉnh sửa từ (Word Correction)

- **Mục tiêu:** Sửa lỗi chính tả, lỗi gõ phím, hoặc các biến thể không chuẩn của từ để đảm bảo tính nhất quán của dữ liệu.
- **Tầm quan trọng:** Lỗi chính tả có thể tạo ra nhiều "từ" khác nhau cho cùng một khái niệm, làm giảm hiệu quả của các mô hình.
- **Các loại lỗi:**
 - Lỗi gõ phím: "hte" thay vì "the", "thich" thay vì "thích".
 - Lỗi chính tả: "chính tar" thay vì "chính tả".
 - Viết tắt không chuẩn: "ko" thay vì "không", "bt" thay vì "biết".

2.4. Chỉnh sửa từ (Word Correction)

Phương pháp:

- Dựa trên từ điển: So sánh từ với một từ điển chuẩn, tìm từ gần nhất (ví dụ: khoảng cách Levenshtein).
- Dựa trên mô hình ngôn ngữ: Sử dụng các mô hình thống kê hoặc học sâu để dự đoán từ đúng dựa trên ngữ cảnh.
- Rule-based: Áp dụng các quy tắc để sửa các lỗi phổ biến.

Ví dụ:

- Văn bản gốc: "Cuốn sách nay rat hay, toi da doc no 2 lan."
- Sửa lỗi gõ phím/viết tắt:
 - "sach" -> "sách"
 - "rat" -> "rất"
 - "toi" -> "tôi"
 - "doc" -> "đọc"
- Văn bản sau chỉnh sửa: "Cuốn sách này rất hay, tôi đã đọc nó 2 lần."

2.5 Lấy từ gốc (Stemming & Lemmatization)

- **Mục tiêu:** Giảm các từ có cùng gốc về một dạng cơ bản duy nhất để chúng được coi là một thực thể duy nhất.
- **Tầm quan trọng:** Giảm kích thước từ điển, tăng cường khả năng tổng quát hóa của mô hình.
- **Ví dụ:**
 - "chơi", "chơi đùa", "đã chơi", "đang chơi" -> "chơi"
 - "running", "runs", "ran" -> "run"

2.5.1 Lấy từ gốc - Stemming

- **Khái niệm:** Là quá trình cắt bỏ các hậu tố (và đôi khi tiền tố) từ một từ để đưa nó về dạng "gốc" (stem).
- **Cách hoạt động:** Thường dựa trên các quy tắc heuristic (quy tắc kinh nghiệm) và không đảm bảo từ gốc tạo ra là một từ có nghĩa trong từ điển.
- **Ưu điểm:** Nhanh chóng, đơn giản.
- **Nhược điểm:** Có thể tạo ra các từ không có nghĩa hoặc cắt quá mức (over-stemming) hoặc cắt không đủ (under-stemming).

2.5.1 Ví dụ Stemming

- Tiếng Anh (Porter Stemmer/LancasterStemmer/SnowballStemmer):
 - "connection" -> "connect"
 - "connecting" -> "connect"
 - "connected" -> "connect"
 - "histories" -> "histori" (không phải từ có nghĩa)
 - "beautiful" -> "beauti" (không phải từ có nghĩa)
- Tiếng Việt (thách thức):
 - Tiếng Việt là ngôn ngữ đơn lập, ít biến đổi hình thái. Stemming không phổ biến và không hiệu quả bằng Lemmatization.
 - Tuy nhiên, có thể áp dụng cho một số trường hợp như loại bỏ dấu câu, hoặc các hậu tố/tiền tố không mang nghĩa.
Ví dụ: "chơi đùa" -> "chơi", "ăn uống" -> "ăn" (đây thực chất là xử lý từ ghép hơn là stemming hình thái).

2.5.2 Lấy từ gốc - Lemmatization

- **Lemmatization** là quá trình đưa một từ về dạng gốc có nghĩa trong từ điển (lemma).
- **Cách hoạt động:** Thường yêu cầu kiến thức về ngữ pháp và từ loại của từ. Sử dụng từ điển và các quy tắc hình thái học phức tạp hơn.
- **Ưu điểm:** Tạo ra các từ gốc có nghĩa, chính xác hơn Stemming.
- **Nhược điểm:** Chậm hơn Stemming, phức tạp hơn.

2.5.2 Lấy từ gốc - Lemmatization

- Tiếng Anh (WordNetLemmatizer):

"running" -> "run"

"runs" -> "run"

"ran" -> "run"

"better" -> "good" (xử lý dạng so sánh hơn)

"geese" -> "goose" (xử lý dạng số nhiều bất quy tắc)

- Tiếng Việt:

- Do tiếng Việt là ngôn ngữ đơn lập, Lemmatization thường tập trung vào việc đưa các từ ghép về từ gốc hoặc chuẩn hóa các biến thể.
- Ví dụ: "học sinh" -> "học_sinh" (giữ nguyên từ ghép nếu có nghĩa), hoặc "ăn uống" -> "ăn" (nếu "uống" được coi là bổ sung).
- Thường kết hợp với việc gán nhãn từ loại để xác định chính xác dạng gốc.

2.5.3 So sánh Stemming và Lemmatization

Đặc điểm	Stemming	Lemmatization
Mục tiêu	Cắt bỏ hậu tố để có dạng "gốc" nhanh chóng.	Đưa về dạng từ điển có nghĩa (lemma).
Kết quả	Có thể là từ không có nghĩa.	Luôn là từ có nghĩa trong từ điển.
Độ chính xác	Thấp hơn.	Cao hơn.
Tốc độ	Nhanh hơn.	Chậm hơn.
Phức tạp	Đơn giản, dựa trên quy tắc heuristic.	Phức tạp, yêu cầu từ điển và kiến thức ngữ pháp.
Ứng dụng	Tìm kiếm thông tin (Information Retrieval), spam filtering.	Phân tích ngữ nghĩa, dịch máy, chatbot.

Bài tập 2

Hãy viết code trên python để thực hiện chỉnh sửa từ gốc sử dụng stemming và lemmatization cho câu sau bằng 2 thư viện khác nhau (cho mỗi kỹ thuật)

- “Hnay là thứ tu. Chúng tôi có tiết học về phân tích và xu li dữ liệu văn bản. Tiếtt học khá thú viii, tuy nhiên vẫn còn nhiều thứ chưa hiểu hết.....”

3 Hiểu cấu trúc ngữ pháp

- **Khái niệm:** Phân tích cấu trúc ngữ pháp (Syntactic Analysis) là quá trình phân tích câu để hiểu cấu trúc ngữ pháp của nó, mối quan hệ giữa các từ và cách chúng tạo thành các cụm từ, mệnh đề.
- **Mục tiêu:**
 - Hiểu được ý nghĩa chính xác của câu (ngữ nghĩa).
 - Phát hiện các lỗi ngữ pháp.
 - Hỗ trợ các tác vụ NLP phức tạp hơn như dịch máy, tóm tắt văn bản, trả lời câu hỏi.
- **Các cấp độ phân tích:** Từ từ loại, cụm từ đến cấu trúc câu hoàn chỉnh.

3.1 Gán nhãn từ loại

- **Khái niệm:** Là quá trình gán nhãn từ loại (ví dụ: danh từ, động từ, tính từ, trạng từ) cho mỗi từ trong một câu.
- **Tầm quan trọng:**
 - Là bước nền tảng cho nhiều tác vụ NLP khác.
 - Giúp phân biệt các từ có cùng cách viết nhưng khác từ loại (ví dụ: "book" vừa là danh từ, vừa là động từ).
 - Hỗ trợ cho Lemmatization, phân tích cú pháp.
- **Nhãn từ loại:** Thường theo một bộ nhãn chuẩn (ví dụ: Penn Treebank Tagset cho tiếng Anh, hoặc các bộ nhãn cho tiếng Việt).

3.1 Gán nhãn từ loại

- **Văn bản:** "Tôi thích ăn phở Hà Nội."
- **Kết quả POS Tagging (ví dụ):**
 - Tôi: Đại từ (Pronoun - PR)
 - thích: Động từ (Verb - V)
 - ăn: Động từ (Verb - V)
 - phở: Danh từ (Noun - N)
 - Hà Nội: Danh từ riêng (Proper Noun - Np)
 - "." : Dấu câu (Punctuation - PUNC)
- **Kết quả đầy đủ:** "Tôi/PR thích/V ăn/V phở/N Hà_Nội/Np ./PUNC"
- **Lưu ý:** Đối với tiếng Việt, từ loại thường được gán cho cả từ ghép ("thành phố", "Việt Nam").

3.2 Phân tích cú pháp nông

Chunking (còn gọi là *shallow parsing* – *phân tích cú pháp nông*) là bước **nhóm các từ đơn lẻ thành cụm từ có ý nghĩa ngữ pháp** (phrase), dựa trên kết quả gán nhãn từ loại (POS tagging).

- POS Tagging

Cô/NN gái/NN trẻ/JJ đang/VB đọc/VB một/DT cuốn/NN sách/NN thú_vị/JJ ./.

- Chunking ?

3.2 Phân tích cú pháp nông

- "Cô gái trẻ đang đọc một cuốn sách thú vị."
- Kết quả Chunking:
[NP Cô gái trẻ] [VP đang đọc] [NP một cuốn sách thú vị].
- Giải thích:
- [NP Cô gái trẻ] là một cụm danh từ.
- [VP đang đọc] là một cụm động từ.
- [NP một cuốn sách thú vị] là một cụm danh từ khác.
- **Ứng dụng:** Trích xuất thông tin (ví dụ: tìm tất cả các cụm danh từ trong văn bản).

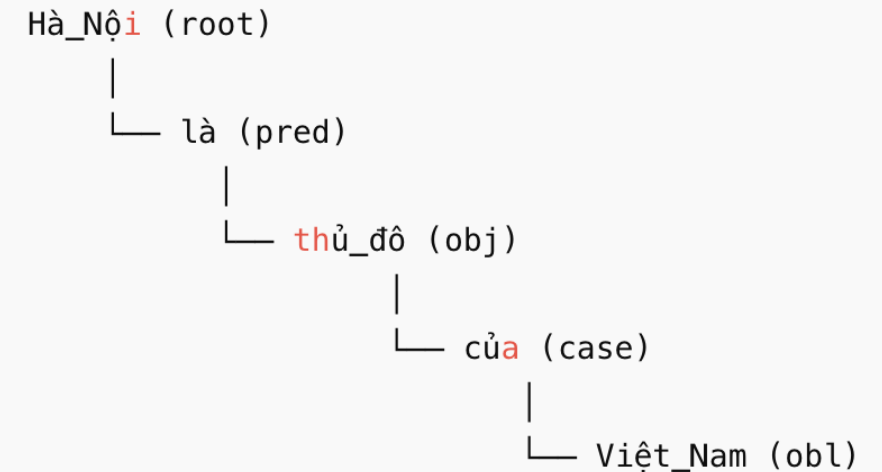
3.3 Phân tích cú pháp phụ thuộc (Dependency Parsing)

- **Phân tích cú pháp phụ thuộc** (*Dependency Parsing*) là quá trình phân tích mối quan hệ phụ thuộc ngữ pháp giữa các từ trong một câu, xác định từ nào là "**đầu**" (head) và từ nào là "**phụ thuộc**" (dependent) của nó.
- **Mục tiêu:** Biểu diễn cấu trúc ngữ pháp dưới dạng một đồ thị có hướng, nơi các nút là từ và các cạnh là mối quan hệ phụ thuộc.
- **Ưu điểm:**
 - Tập trung vào mối quan hệ ngữ nghĩa giữa các từ.
 - Thường hiệu quả hơn cho các ngôn ngữ có cấu trúc câu linh hoạt (như tiếng Việt).

3.3 Phân tích cú pháp phụ thuộc

- Văn bản: "**Hà Nội là thủ đô của Việt Nam.**"
- Mỗi quan hệ phụ thuộc (ví dụ):
 - **Hà Nội** → gốc (root) của câu.
 - **là** → phụ thuộc vào **Hà Nội** (quan hệ **pred** – vị ngữ).
 - **thủ đô** → phụ thuộc vào **là** (quan hệ **obj** – bổ ngữ).
 - **của** → phụ thuộc vào **thủ đô** (quan hệ **case/mark** – giới từ).
 - **Việt Nam** → phụ thuộc vào **của** (quan hệ **obl** – bổ ngữ của giới từ).
 - Dấu chấm "." → phụ thuộc vào root (quan hệ **punct**).
- Ứng dụng: Trích xuất quan hệ (ví dụ: "thủ đô của Việt Nam"), dịch máy, trả lời câu hỏi.

- Biểu diễn đồ thị:



3.4 Phân tích cú pháp thành phần

- **Phân tích cú pháp thành phần** là quá trình phân tích câu thành một cấu trúc cây phân cấp (parse tree) dựa trên các thành phần ngữ pháp (phrases, clauses).
- **Mục tiêu:** Xác định cách các từ nhóm lại với nhau để tạo thành các cụm từ, và cách các cụm từ này kết hợp để tạo thành câu.
- **Ưu điểm:** Cung cấp biểu diễn ngữ pháp chi tiết và đầy đủ nhất.
- **Nhược điểm:** Phức tạp và tốn kém tài nguyên tính toán hơn các phương pháp khác.

3.4 Phân tích cú pháp thành phần

- **Văn bản:** "Cô gái trẻ đang đọc một cuốn sách thú vị."
 - **Cây cú pháp (Constituency Tree) - Minh họa:**
 - (S
 - (NP (NNP Cô) (NNP gái) (JJ trẻ))
 - (VP (VBP đang)
 - (VP (VB đọc)
 - (NP (DT một) (NN cuốn) (NN sách) (JJ thú vị))))
 - (. .))
 - S: Sentence (Câu)
 - NP: Noun Phrase (Cụm danh từ)
 - VP: Verb Phrase (Cụm động từ)
 - NNP: Proper Noun (Danh từ riêng)
 - JJ: Adjective (Tính từ)
 - VBP: Verb, non-3rd person singular present (Động từ)
 - VB: Verb, base form (Động từ nguyên thể)
 - DT: Determiner (Mạo từ/Từ hạn định)
- **Ứng dụng:** Dịch máy, tóm tắt văn bản, phân tích ngữ nghĩa sâu.

Bài tập 3

Cho câu sau: *“Các sinh viên Khoa học máy tính ở PTIT đang học phân tích dữ liệu tại cơ sở Ngọc Trục.”*

Hãy viết chương trình trên python thực hiện các yêu cầu sau:

1. Gán nhãn từ loại (POS Tagging) cho từng từ.
2. Thực hiện phân tích cú pháp nông (Chunking), tìm các cụm danh từ (NP), cụm động từ (VP).
3. Phân tích cú pháp phụ thuộc (Dependency Parsing), liệt kê quan hệ phụ thuộc giữa các từ.
4. Phân tích cú pháp thành phần (Constituency Parsing), biểu diễn cây thành phần của câu.
5. (Tùy chọn) Vẽ trực quan cây phụ thuộc hoặc cây thành phần.